

Max Heap



MAX HEAP

"Write a C program to perform the following operations:
Insert the employee salaries: 45000, 52000, 61000, 48000, 75000, 68000, 59000. Display the Max Heap after every insertion
Delete the highest salary from the heap. Display the heap after deletion"

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int heap[20];
6     int n = 0;
7     int a[] = {45000, 52000, 61000, 48000, 75000, 68000, 59000};
8     int i, j, temp;
9
10    for (i = 0; i < 7; i++)
11    {
12        /* Insert */
13        heap[n] = a[i];
14        n++;
15
16        /* Heapify up */
17        j = n - 1;
18
19        while (j > 0)
20        {
21            int parent = (j - 1) / 2;
```

OUTPUT

Absolute system paths (e.g. C:\) are not allowed.

Final Priority Queue ▾

FINAL PRIORITY QUEUE

Write a C program to implement the following operations:
Insert student requests with priorities:
(101, 3), (102, 1), (103, 5), (104, 2), (105, 4)
Display the queue elements in priority order
Delete the highest priority request
Insert a new request ▾

```
1 #include <stdio.h>
2
3 struct Request {
4     int id;
5     int priority;
6 };
7
8 int main()
9 {
10     struct Request q[10] = {
11         {101, 3},
12         {102, 1},
13         {103, 5},
14         {104, 2},
15         {105, 4}
16     };
17
18     int n = 5;
19     int i, j;
20     struct Request temp;
21
```

OUTPUT

Priority Queue:

ID: 103 Priority: 5

ID: 105 Priority: 4

ID: 101 Priority: 3

ID: 104 Priority: 2

ID: 102 Priority: 1

Deleted request: 103

Final Priority Queue:

Heap Sort

HEAP SORT

"Write a C program to implement Heap Sort for the following set of numbers:

45, 12, 78, 34, 23, 89, 67, 10, 56, 99

The program should:

Construct the Max

Heap

Display the heap after each adjustment

Sort the elements in ascending order using

```
1 #include <stdio.h>
2
3 void display(int a[], int n)
4 {
5     int i;
6
7     for (i = 0; i < n; i++)
8         printf("%d ", a[i]);
9
10    printf("\n");
11 }
12
13 void heapify(int a[], int n, int i)
14 {
15     int largest, left, right, temp;
16
17     largest = i;
18     left = 2 * i + 1;
19     right = 2 * i + 2;
20
21     if (left < n && a[left] > a[largest])
```

OUTPUT

```
Max Heap:
99 56 89 45 23 78 67 10 34 12
After adjustment:
89 56 78 45 23 12 67 10 34 99
After adjustment:
78 56 67 45 23 12 34 10 89 99
After adjustment:
67 56 34 45 23 12 10 78 89 99
After adjustment:
56 45 34 10 23 12 67 78 89 99
```

Heap and Priority Queue

HEAP AND PRIORITY QUEUE

Write a C program to implement all Heap and Priority Queue operations through a menu-driven program."

```
1 #include <stdio.h>
2
3 int heap[20], n = 0;
4
5 void insert(int x)
6 {
7     int i, temp;
8
9     heap[n] = x;
10    i = n;
11    n++;
12
13    while (i > 0 && heap[i] > heap[(i - 1) / 2])
14    {
15        temp = heap[i];
16        heap[i] = heap[(i - 1) / 2];
17        heap[(i - 1) / 2] = temp;
18
19        i = (i - 1) / 2;
20    }
21 }
```

OUTPUT

```
Heap: 50
Heap: 50 30
Heap: 70 30 50
Heap: 70 40 50 30
Deleted: 70
Heap: 50 40 30
```

Heap Property



HEAP PROPERTY

Write a C program to check whether a given binary tree satisfies Heap property.

```
1 #include <stdio.h>
2
3 int isMaxHeap(int a[], int n)
4 {
5     int i;
6
7     for (i = 0; i <= (n - 2) / 2; i++)
8     {
9         if (2 * i + 1 < n && a[i] < a[2 * i + 1])
10             return 0;
11
12         if (2 * i + 2 < n && a[i] < a[2 * i + 2])
13             return 0;
14     }
15
16     return 1;
17 }
18
19 int main()
20 {
21     int a[] = {90, 70, 80, 40, 50, 60, 30};
```


OUTPUT

```
The binary tree satisfies Max Heap property
```

Heap Operations ▾

HEAP OPERATIONS

"Write a menu-driven C program to implement all Heap operations for the following elements: 18, 25, 40, 12, 30, 50, 10, 45 and perform the following operations :
Heap creation ,
Insertion, Deletion, Heap display and Searching an element"

```
1 #include <stdio.h>
2
3 int h[20], n = 0;
4
5 void insert(int x)
6 {
7     int i, t;
8
9     i = n;
10    h[n++] = x;
11
12    while (i > 0 && h[i] > h[(i - 1) / 2])
13    {
14        t = h[i];
15        h[i] = h[(i - 1) / 2];
16        h[(i - 1) / 2] = t;
17        i = (i - 1) / 2;
18    }
19 }
20
21 void del()
```

OUTPUT

```
Heap: 50 45 40 30 18 25 10 12  
Search 30: Found  
Deleted: 50  
After deletion: 45 30 40 12 18 25 10
```

MERGING HEAPS

Write a C program to merge two Max Heaps into a single Heap.

```
1 #include <stdio.h>
2
3 void heapify(int a[], int n, int i)
4 {
5     int largest = i;
6     int left = 2*i + 1;
7     int right = 2*i + 2;
8     int temp;
9
10    if (left < n && a[left] > a[largest])
11        largest = left;
12
13    if (right < n && a[right] > a[largest])
14        largest = right;
15
16    if (largest != i)
17    {
18        temp = a[i];
19        a[i] = a[largest];
20        a[largest] = temp;
21    }
```

OUTPUT

```
Merged Max Heap:
50 45 40 30 20 35
```

Priority Queue



PRIORITY QUEUE

"Write a C program to perform the following operations using priority queue: Insert patient records with severity levels:
Ravi-4, Kumar-2, Priya-5, Anu-1, Deepak-3
Display the patients in priority order
Treat and remove the highest priority patient"

```
1 #include <stdio.h>
2
3 struct Patient {
4     char name[20];
5     int priority;
6 };
7
8 int main()
9 {
10     struct Patient p[10] = {
11         {"Ravi", 4},
12         {"Kumar", 2},
13         {"Priya", 5},
14         {"Anu", 1},
15         {"Deepak", 3}
16     };
17
18     int n = 5;
19     int i, j;
20     struct Patient temp;
21
```

OUTPUT

Priority Queue:

Priya - 5

Ravi - 4

Deepak - 3

Kumar - 2

Anu - 1

Treated: Priya

Updated Priority Queue:

Heap- Min & Max ▾

HEAP- MIN & MAX ELEMENT

Write a C program to find the maximum and minimum element using Heap.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[10] = {45, 20, 60, 10, 30, 50, 70};
6     int n = 7;
7     int i, min, max;
8
9     min = a[0];
10    max = a[0];
11
12    for (i = 1; i < n; i++)
13    {
14        if (a[i] < min)
15            min = a[i];
16
17        if (a[i] > max)
18            max = a[i];
19    }
20
21    printf("Minimum element = %d\n", min);
```

OUTPUT

```
Minimum element = 10
Maximum element = 70
```

Min Heap- Operati ▾

MIN HEAP- OPERATIONS

Write a C program to construct a Min Heap using the following data: 55, 20, 35, 10, 90, 40, 15, 25 and perform the following operations:
Insert elements one by one into the Min Heap
Display the heap after every insertion
Delete the minimum element

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int h[20], n = 0;
6     int a[] = {55, 20, 35, 10, 90, 40, 15, 25};
7     int i, j, temp;
8
9     /* Insert into Min Heap */
10    for (i = 0; i < 8; i++)
11    {
12        h[n] = a[i];
13        j = n;
14        n++;
15
16        while (j > 0 && h[j] < h[(j - 1) / 2])
17        {
18            temp = h[j];
19            h[j] = h[(j - 1) / 2];
20            h[(j - 1) / 2] = temp;
21            j = (j - 1) / 2;
```


OUTPUT

```
After inserting 55: 55
After inserting 20: 20 55
After inserting 35: 20 55 35
After inserting 10: 10 20 35 55
After inserting 90: 10 20 35 55 90
After inserting 40: 10 20 35 55 90 40
After inserting 15: 10 20 15 55 90 40 35
After inserting 25: 10 20 15 25 90 40 35 55
```

```
Deleted minimum: 10
```